OS ORANGE LEVEL REPORT

NAME: CHARAN M

SRN: PES1UG24CS125

PHASE 1 SCREENSHOT 1A

```
charan@charan-VirtualBox:~/OS-ORANGE$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
charan@charan-VirtualBox:~/OS-ORANGE$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

SCREENSHOT 1B

```
charan@charan-VirtualBox:~/OS-ORANGE$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

PHASE 2

SCREENSHOT 2A

```
charan@charan-VirtualBox:~/OS-ORANGE$ make test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
charan@charan-VirtualBox:~/OS-ORANGE$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

SCREENSHOT 2B

```
charan@charan-VirtualBox:~/OS-ORANGE$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

PHASE 3

SCREENSHOT 3A

```
charan@charan-VirtualBox:~/OS-ORANGE$ rm -rf .pes
charan@charan-VirtualBox:~/OS-ORANGE$ ./pes init
Initialized empty PES repository in .pes/
charan@charan-VirtualBox:~/OS-ORANGE$ echo "hello" > file1.txt
charan@charan-VirtualBox:~/OS-ORANGE$ echo "world" > file2.txt
charan@charan-VirtualBox:~/OS-ORANGE$ ./pes add file1.txt file2.txt
charan@charan-VirtualBox:~/OS-ORANGE$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  Makefile
  untracked:  index.c
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  commit.h
  untracked:  commit.c
  untracked:  test_tree.c
  untracked:  tree.h
  untracked:  tree.c
  untracked:  index.h
  untracked:  README.md
  untracked:  pes.h
  untracked:  object.c
  untracked:  test_objects
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  test_objects.c
```

SCREENSHOT 3B

```
charan@charan-VirtualBox:~/OS-ORANGE$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776667303 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776667312 6 file2.txt
```

PHASE 4

SCREENSHOT 4A

```
charan@charan-VirtualBox:~/OS-ORANGE$ ./pes log
commit 0720c6b7238ea92bc3b77a6ff54ea29b2156931da154fbcd410443c3eadc605c
Author: PES User <pes@localhost>
Date:   1776692287

    Add farewell

commit a9d375c8108a62c9fdd82a1d2f1ed4ce882efc9fa293efbce60357cfbf28e6f8
Author: PES User <pes@localhost>
Date:   1776692251

    Add worldell

commit 712cddf970c34482a8d94e349fab0f5234ebae35f0607f2d4ea70292bb6b8124
Author: PES User <pes@localhost>
Date:   1776692227

    Initial committial commiQ
```

SCREENSHOT 4B

```
charan@charan-VirtualBox:~/OS-ORANGE$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/07/20c6b7238ea92bc3b77a6ff54ea29b2156931da154fbcd410443c3eadc605c
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/71/2cddf970c34482a8d94e349fab0f5234ebae35f0607f2d4ea70292bb6b8124
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/a9/d375c8108a62c9fdd82a1d2f1ed4ce882efc9fa293efbce60357cfbf28e6f8
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

SCREENSHOT 4C

```
charan@charan-VirtualBox:~/OS-ORANGE$ cat .pes/refs/heads/main
0720c6b7238ea92bc3b77a6ff54ea29b2156931da154fbcd410443c3eadc605c
charan@charan-VirtualBox:~/OS-ORANGE$ cat .pes/HEAD
ref: refs/heads/main
```

Make integration

```
charan@charan-VirtualBox:~/OS-ORANGE$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 1e1dd38d2d8c... Initial commit
```

```
Log after first commit:
commit 1e1dd38d2d8cbe45a82a3505fb602c3b21d153d5fe97455ab7eb73c2e301c917
Author: PES User <pes@localhost>
Date:   1776692446

    Initial commit


--- Second Commit ---
Committed: 15f2795a306a... Update file.txt

--- Third Commit ---
Committed: 91d528c3a4c7... Add farewell

--- Full History ---
commit 91d528c3a4c7d8e70aa0dda8a7ce812628e49f23507788a7e09c22d99487eb5a
Author: PES User <pes@localhost>
Date:   1776692446

    Add farewell

commit 15f2795a306adf155543a142a4e18abc521681bdff69bd61eb769b776a8b1396
Author: PES User <pes@localhost>
Date:   1776692446

    Update file.txt

commit 1e1dd38d2d8cbe45a82a3505fb602c3b21d153d5fe97455ab7eb73c2e301c917
Author: PES User <pes@localhost>
Date:   1776692446
```

```
    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
91d528c3a4c7d8e70aa0dda8a7ce812628e49f23507788a7e09c22d99487eb5a

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/15/f2795a306adf155543a142a4e18abc521681bdff69bd61eb769b776a8b1396
.pes/objects/1e/1dd38d2d8cbe45a82a3505fb602c3b21d153d5fe97455ab7eb73c2e301c917
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/91/d528c3a4c7d8e70aa0dda8a7ce812628e49f23507788a7e09c22d99487eb5a
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
```

PHASE 5 & 6 ANSWERS

Q5.1) A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

pes checkout <branch> updates .pes/HEAD to point to the new branch, reads the commit hash from .pes/refs/heads/<branch>, loads its tree, and updates the working directory to match that snapshot. The index is also updated accordingly.
The complexity comes from safely modifying the working directory without overwriting uncommitted changes and correctly handling file additions, deletions, and directories.

Q5.2) When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Compare working directory with index to detect modified files. Then compare those files with the target branch’s tree.
If a file is modified locally and also different in the target branch, checkout must stop to avoid overwriting changes.

Q5.3) "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In detached HEAD, commits are created but not attached to any branch, so they can become unreachable.
To recover, create a branch pointing to that commit before it gets garbage collected.

Q6.1) Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Start from all branch heads and traverse commits, trees, and blobs using a graph traversal (like DFS/BFS). Store visited hashes in a hash set.
Anything not visited is unreachable and can be deleted.
For 100,000 commits, you may visit hundreds of thousands of objects including trees and blobs.

Q6.2) Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

If GC runs during a commit, it may delete objects that are being created but not yet referenced. This causes data loss.
Git avoids this by using locks and by not deleting recently created objects, ensuring commits complete safely before GC removes anything.